Retail Analytics & Machine Learning Project

Churn Prediction, Future Monetary Value Estimation, and Customer Segmentation

Author: *Chayma Dallel*
(GI2 S3)

Date: April 25, 2026

Abstract

This project aims to predict and analyze customer behavior for a retail website through three complementary lenses: (i) a **classification** task to predict churn (at-risk customers), (ii) a **regression** task to estimate future monetary value (using `MonetaryTotal` as a proxy), and (iii) a **clustering** task to segment customers into homogeneous groups.

The proposed solution is structured as an industrial-grade pipeline: data ingestion, preprocessing, training, evaluation, and deployment via a Web application (Flask) and/or a Streamlit interface.

The dataset consists of 4,372 customers and 52 variables (numerical and categorical). The modules generate EDA reports, aligned train/test sets, serialized models, and comprehensive visualizations.

Contents

1 Context and Objectives **3**
- 1.1 Business Problem . . . 3
- 1.2 Machine Learning Objectives . . . 3

2 Data Overview **3**
- 2.1 Source and Dimensions . . . 3
- 2.2 Data Quality and EDA Findings . . . 3
- 2.3 Key Variables . . . 4

3 Software Architecture **4**
- 3.1 Modular Organization . . . 4
- 3.2 Directory Structure (Artifacts) . . . 4

4 Preprocessing Pipeline **4**
- 4.1 Core Stages . . . 4
- 4.2 Data Leakage Management . . . 5
- 4.3 Production Robustness: Input Schema . . . 5

5 Models and Methodology **5**
- 5.1 Train/Test Split . . . 5
- 5.2 Classification: Churn Prediction . . . 5
- 5.3 Regression: Monetary Value Estimation . . . 6
- 5.4 Clustering: Customer Segmentation . . . 6

6 Evaluation and Optimization **6**
- 6.1 Hyperparameter Tuning . . . 6
- 6.2 Consistency Checks . . . 6

7 Prediction (Inference) **6**
- 7.1 Test Mode . . . 6
- 7.2 Sample Mode (New Customers) . . . 7

8 Deployment: Web Application **7**
- 8.1 Flask Framework . . . 7
- 8.2 User Interface . . . 7
- 8.3 UI Limitations . . . 7

9 Results and Deliverables **7**
- 9.1 Generated Files . . . 7
- 9.2 Visualizations . . . 8

10 Discussion: Technical Choices, Limits, and Future Work **8**
- 10.1 Technical Choices . . . 8
- 10.2 Limitations . . . 8

10.3 Recommended Improvements 8

11 Execution Guide **9**

11.1 Recommended Workflow 9

11.2 Reproducibility 9

12 Appendix: Code Snippets (Optional) **9**

12.1 Example: Model Serialization 9

12.2 Example: Pipeline Concept 9

1 Context and Objectives

1.1 Business Problem

In a retail context, the ability to anticipate customer loss (churn) and measure future value is critical. A data-driven approach enables:

- **Retention Prioritization** via targeted campaigns for high-risk customers.
- **Value Optimization** through upsell/cross-sell opportunities for high-value customers.
- **Customer Segmentation** for personalized marketing and tailored offers.

1.2 Machine Learning Objectives

The project encompasses three primary tasks:

1. **Classification (Churn)**: Predict the binary target variable `Churn`.
2. **Regression (Monetary)**: Predict the continuous variable `MonetaryTotal`.
3. **Clustering (Segmentation)**: Segment customers using the `KMeans` algorithm.

2 Data Overview

2.1 Source and Dimensions

- **Dataset**: `retail_customers_COMPLETE_CATEGORICAL.csv`
- **Row Count**: 4,372
- **Column Count**: 52

2.2 Data Quality and EDA Findings

Exploratory Data Analysis (EDA) and generated reports highlight the following:

- **Duplicates**: 0
- **Missing Values**: Predominantly found in `Age` and `AvgDaysBetweenPurchases`.
- **Suspicious Values**: Observed values such as `-1, 99, 999, Unknown, NA, ""` in specific variables.

2.3 Key Variables

Typical variables oriented around RFM (Recency, Frequency, Monetary) and customer profiles:

- **RFM**: `Recency`, `Frequency`, `MonetaryTotal`.
- **Demographics**: `Age`, `Gender`, `Region`.
- **Engagement**: `SatisfactionScore`, preferences, and loyalty status.

3 Software Architecture

3.1 Modular Organization

The project is structured into modules aligned with the ML lifecycle:

- **Preprocessing**: `src/preprocessing.py`
- **Training**: `src/train_model.py`
- **Evaluation**: `src/evaluation.py`
- **Prediction**: `src/predict.py`
- **Utilities**: `src/utils.py`
- **Feature Engineering**: `src/features.py` (Scikit-learn transformer)
- **Web Deployment**: `app.py` + HTML templates

3.2 Directory Structure (Artifacts)

- `data/raw/`: Raw datasets.
- `data/train_test/`: Transformed sets (`X_train_pca.csv`, etc.).
- `data/predictions/`: Output files (`predictions.csv`).
- `models/`: Serialized models (`.pkl`) and input schema (`raw_schema.json`).
- `reports/`: Logs, EDA reports, plots, VIF analysis, etc.

4 Preprocessing Pipeline

4.1 Core Stages

The preparation pipeline is designed for **production reusability**: a single transformation chain is serialized and applied identically during both training and inference.

1. **Suspicious Value Cleaning**: Replacement of anomalies with `NaN`.
2. **Feature Engineering**:
    - `RegYear`, `RegMonth` extracted from `RegistrationDate`.

- `MonetaryPerDay` $\approx \frac{\texttt{MonetaryTotal}}{\texttt{Recency}+1}$.
- `AvgBasketValue` $\approx \frac{\texttt{MonetaryTotal}}{\texttt{Frequency}}$.
- `TenureRatio` (if `CustomerTenure` is available).

3. **Age Handling**: Values outside the logical range (e.g., 18–81) → `NaN`.
4. **Imputation Strategy**:
   - **Numerical**: Median.
   - **Categorical**: Most frequent modality (Mode).
5. **Encoding**: OneHotEncoder (`handle_unknown=ignore`).
6. **Scaling**: StandardScaler for numerical features.
7. **Dimensionality Reduction**: PCA (`n_components=10`).

4.2 Data Leakage Management

Certain variables may encapsulate information too closely related to the target (e.g., `ChurnRisk`). These are removed from the pipeline via a specific `drop_cols` list to ensure model integrity.

4.3 Production Robustness: Input Schema

A `models/raw_schema.json` file saves the list of expected columns for inference. In the case of partial input (Web forms), the DataFrame is **reindexed**: missing columns are filled with `NaN` and then processed through the imputer.

5 Models and Methodology

5.1 Train/Test Split

The data is split into **80% Training / 20% Testing** with stratification on the churn target:

$$(X_{train}, X_{test}, y^{cls}_{train}, y^{cls}_{test}, y^{reg}_{train}, y^{reg}_{test})$$

5.2 Classification: Churn Prediction

Model Selection **XGBoost Classifier** is utilized for its superior performance on tabular data and its ability to capture non-linear relationships.

Output

- `Churn_Probability`: Probability $\in [0, 1]$.
- `Churn_Predicted`: Default threshold at 0.5 (adjustable).

Metrics Accuracy, AUC ROC, Precision, Recall, and F1-score.

5.3 Regression: Monetary Value Estimation

Model Selection **RandomForestRegressor** is employed to model non-linear relationships while maintaining resistance to noise.

Metrics RMSE, R^2, and visual analysis via `Actual vs. Predicted` scatter plots.

5.4 Clustering: Customer Segmentation

Model Selection **KMeans** is applied to segment customers into homogeneous groups within the PCA-transformed space.

Determining k The **Elbow Method** is calculated for $k \in [2, 10]$. The optimal k is selected via heuristic or the "elbow point."

Output `cluster`: Segment identifier (0, 1, 2, ...).

6 Evaluation and Optimization

6.1 Hyperparameter Tuning

The `evaluation.py` module performs hyperparameter optimization for XGBoost via `GridSearchCV`.

Optimization Metric The primary recommended score is **ROC AUC**, as churn is often imbalanced and the cost of errors is asymmetric.

6.2 Consistency Checks

The scripts include validation steps to prevent:

- Mismatch between `X` and `y` sizes.
- Double-transformation of data.
- Schema inconsistencies between training and inference phases.

7 Prediction (Inference)

7.1 Test Mode

In **test mode**, models are applied to `X_test_pca.csv`. It saves a `predictions.csv` file containing:

- Churn probability.
- Predicted churn vs. Actual churn.
- Predicted monetary value vs. Actual value.
- Assigned cluster.

7.2 Sample Mode (New Customers)

The **sample mode** simulates new customers by sampling raw data and applying:

$$X_{raw} \xrightarrow{\text{pipeline}} X_{pca} \xrightarrow{\text{models}} \hat{y}$$

This ensures the transformation chain remains identical to the one used during training.

8 Deployment: Web Application

8.1 Flask Framework

A Flask application exposes three primary routes:

- `/predict_classification`
- `/predict_regression`
- `/predict_clustering`

8.2 User Interface

The HTML interface (form) allows users to input key variables (e.g., `Age`, `SatisfactionScore`). The pipeline fills in missing variables via the schema, then applies imputation and transformations.

8.3 UI Limitations

For maximum production accuracy, it is recommended to:

- Expose more input variables in the form.
- Connect the API to a database containing the full history of customer features.

9 Results and Deliverables

9.1 Generated Files

- **Data**: `X_train_pca.csv`, `X_test_pca.csv`, `y_train.csv`, etc.
- **Models**: `churn_classifier.pkl`, `monetary_regressor.pkl`, `customer_clusters.pkl`.
- **Pipeline**: `preprocessing_pipeline.pkl`.
- **Schema**: `raw_schema.json`.
- **Reports**: `data_report.txt`, EDA plots, elbow curve, regression scatter plots.

9.2 Visualizations

The following charts are typically produced:

- Histograms (RFM, Age, etc.).
- Correlation Heatmaps.
- Regression Scatter Plots (Actual vs. Predicted).
- Elbow Curve (KMeans).
- 2D Cluster Projections (PC1 vs. PC2).

10 Discussion: Technical Choices, Limits, and Future Work

10.1 Technical Choices

- **Scikit-learn Pipeline**: Ensures reproducibility and consistency.
- **XGBoost**: High performance on tabular datasets.
- **RandomForestRegressor**: A robust and interpretable baseline.
- **KMeans**: Simple and actionable segmentation.

10.2 Limitations

- **PCA**: Reduces interpretability as features are transformed into principal components.
- **0.5 Threshold**: Not optimized for business costs; should be calibrated based on financial impact.
- **Clustering**: KMeans assumes spherical clusters and relies on Euclidean distance.

10.3 Recommended Improvements

1. **Imbalance Management**: Class weights, SMOTE, or Focal Loss.
2. **Probability Calibration**: Platt scaling or Isotonic regression.
3. **Explainability**: Implementation of SHAP to explain churn drivers.
4. **Optimal k Selection**: Silhouette score or Davies-Bouldin index.
5. **Monitoring**: Implementation of Data Drift and Performance Drift logs.
6. **CI/CD**: Unit tests for schema consistency and pipeline reproducibility.

11 Execution Guide

11.1 Recommended Workflow

```
# 1) Preprocessing + Artifact saving
python src/preprocessing.py

# 2) Model Training
python src/train_model.py

# 3) Prediction (Validation on test set)
python src/predict.py --mode test

# 4) Launch Flask App
python app.py
```

11.2 Reproducibility

- Set `random_state=42` in all splits and models.
- Maintain library versions (`requirements.txt`).
- Always serialize the pipeline along with the models.

12 Appendix: Code Snippets (Optional)

12.1 Example: Model Serialization

```
from utils import save_model

# Exporting the trained classifier
save_model(clf, "models/churn_classifier.pkl")
```

12.2 Example: Pipeline Concept

```
pipe = Pipeline(steps=[
    ("fe", FeatureEngineer()),
    ("prep", ColumnTransformer([...])),
    ("pca", PCA(n_components=10))
])
```

Conclusion

This project provides a comprehensive, modular, and deployable solution for churn prediction, monetary value estimation, and customer segmentation in the retail sector. Through standardized pipelines, model serialization, and an accessible Web interface, the system ensures a smooth transition from prototype to production. Future enhancements focusing on explainability (SHAP) and business-cost optimization will further increase the strategic value of this tool.